PROJECT BLUEPRINT

AI-Based EV Battery State of Health (SOH) Prediction and Remaining Useful Life (RUL) Estimation

Machine Learning & Data Science System Architecture and Design Document

Course	Introduction to EV & HEV
Document Type	Project Blueprint / Architecture Document
Project Scope	Machine Learning / Data Science pipeline only
Dataset	NASA Li-ion Battery Aging Dataset (B0005, B0006, B0007, B0018)
Status	Pre-implementation design stage

This document defines what will be built, why it is being built, how the system components connect, how the project is divided into stages, and what the inputs and outputs of each stage are. It is a design reference for step-by-step implementation and is not the final project report.

Table of Contents

1. Project Overview 3
2. Problem Statement 3
3. Project Objectives 4
4. Dataset 4
5. Project Division 5
6. Overall System Architecture 6
7. Detailed ML Architecture 7
8. Input and Output Architecture 8
9. SOH Calculation Architecture 9
10. RUL Architecture 9
11. Notebook Architecture 10
12. Technology Stack 12
13. Model Comparison Strategy 13
14. Evaluation Metrics 13
15. Data Leakage Prevention 13
16. Expected Final Project Flow 15
17. Project Scope 15
18. Final Deliverables 16
19. Future Enhancements 16

1. Project Overview

This project builds a machine learning system that studies how an electric vehicle (EV) battery ages over repeated use and predicts two things: its current **State of Health (SOH)** and its **Remaining Useful Life (RUL)**. The system is trained and evaluated using real battery aging data rather than simulated numbers.

EV battery health prediction matters because a battery is the single most expensive and most safety-critical component of an electric vehicle. As batteries are charged and discharged repeatedly, their ability to store energy gradually decreases. If this degradation is not tracked, a vehicle owner or fleet operator has no early warning before the battery becomes unreliable, unsafe, or due for replacement.

What SOH Means

State of Health (SOH) is a percentage that describes how much usable capacity a battery still has compared to when it was brand new. A new battery starts at 100% SOH. As it ages, this value decreases. SOH is the primary indicator of how "healthy" a battery currently is.

What RUL Means

Remaining Useful Life (RUL) estimates how many more charge/discharge cycles (or how much more time) a battery can be used before it reaches an End-of-Life (EOL) condition, typically defined as the point where SOH drops below a chosen threshold (commonly 70-80% of original capacity).

What Problem This Project Solves

The project solves the problem of not knowing a battery's condition until it fails or performs poorly. By learning the relationship between how a battery is used (voltage, current, temperature, cycle count) and how its capacity degrades, the system can estimate SOH directly from usage data and project forward to estimate RUL.

What the Final System Will Produce

At the end of the project, the system will be able to take battery-cycle measurements and produce:

- A predicted SOH value (%) for a given cycle
- A battery health category derived from that SOH value
- A visualization of the battery's SOH degradation trend over cycles
- An estimated End-of-Life (EOL) cycle
- An estimated Remaining Useful Life (RUL), in cycles

2. Problem Statement

Electric vehicle batteries degrade progressively as a result of repeated charging and discharging cycles. This degradation is influenced by several interacting factors:

- **Charging/discharging cycles** — each full cycle causes small, cumulative wear on the internal chemistry
- **Temperature** — high or low operating temperatures accelerate degradation
- **Current** — the magnitude of charge and discharge current affects internal stress on the cell
- **Voltage** — voltage behavior during charge and discharge reflects internal resistance changes
- **Operating conditions** — the overall pattern of usage across the battery's lifetime

Because this degradation is gradual and not directly visible to a vehicle's user, determining a battery's health and its remaining useful life is essential for:

- **Predictive maintenance** — scheduling service before a failure occurs
- **Battery monitoring** — continuously tracking condition over the battery's lifetime
- **EV reliability** — ensuring the vehicle performs as expected throughout its service life
- **Battery replacement planning** — budgeting and planning replacement before end-of-life is reached

Formal Problem Statement

Given historical battery-cycle measurements (voltage, current, temperature, capacity and cycle count) from lithium-ion cells subjected to repeated charge/discharge cycling, the objective of this project is to build a supervised machine learning system that (a) estimates the current State of Health (SOH) of a battery from cycle-level features, and (b) uses the resulting SOH degradation trend to estimate the battery's Remaining Useful Life (RUL) in cycles.

3. Project Objectives

1. Process the NASA Li-ion battery aging dataset (.mat files).
2. Extract useful battery-cycle information from raw charge, discharge and impedance records.
3. Calculate battery SOH from capacity measurements.
4. Analyze battery degradation patterns through exploratory data analysis.
5. Build machine learning models to predict SOH from cycle-level features.
6. Compare multiple regression algorithms under identical conditions.
7. Validate the models using a strategy appropriate for sequential cycle data.
8. Select and tune the best-performing model.
9. Estimate Remaining Useful Life from the SOH degradation trend.
10. Provide interpretable visualizations and a clear summary of results.

4. Dataset

This project uses the **NASA Li-ion Battery Aging Dataset**, a widely used public dataset published by the NASA Prognostics Center of Excellence. It records the behavior of lithium-ion cells that were repeatedly charged and discharged under controlled laboratory conditions until their capacity degraded significantly.

The project uses four battery cells from this dataset:

- B0005
- B0006
- B0007
- B0018

What the Data Represents

Each battery was run through repeated **operational cycles**. A single cycle generally consists of a charge phase, a discharge phase, and (periodically) an impedance measurement phase. Over hundreds of such cycles, the measurable capacity of the battery decreases, which is the underlying signal this project uses to determine SOH.

What a Battery Cycle Means

A cycle is one complete charge-then-discharge operation performed on the cell. Each cycle in the dataset contains time-series measurements recorded throughout that charge or discharge process, rather than a single summary number.

Data Recorded Within the Dataset

- **Charge data** — voltage, current and temperature recorded while the cell is being charged
- **Discharge data** — voltage, current, temperature and measured capacity recorded while the cell is discharging
- **Impedance data** — periodic electrochemical impedance measurements taken between cycles
- **Voltage** — cell terminal voltage over time within a cycle
- **Current** — charge/discharge current applied to the cell over time
- **Temperature** — cell surface temperature recorded over time
- **Capacity** — measured discharge capacity for a given cycle, used to derive SOH
- **Time** — the timestamp axis associated with each in-cycle measurement

Raw Format

The original data for each battery is distributed as a MATLAB `.mat` file containing a nested structure of cycles and measurements. Because this format is not directly usable for machine learning, Phase 2 of this project converts it into a structured, flat tabular dataset (one row per cycle, or per measurement, depending on the stage of processing).

No dataset statistics (such as exact cycle counts, capacity values, or degradation rates) are assumed at this design stage — these will be determined during Phase 1 (Dataset Understanding) once the raw files are inspected.

5. Project Division

The project is divided into ten sequential phases. Each phase produces a well-defined output that feeds into the next phase.

Phase 1 — Dataset Understanding

- Inspect the raw .mat files for each battery
- Understand the nested MATLAB structure
- Understand how battery cycles are organized within the file
- Identify which measurements are actually usable

Phase 2 — Data Extraction & Preprocessing

- Extract charge and discharge information from the nested structure
- Convert the nested data into a flat, tabular form
- Handle missing or invalid readings
- Combine the four battery datasets into a single working dataset

Phase 3 — SOH Calculation & EDA

- Calculate SOH for every cycle
- Analyze capacity degradation across cycles
- Plot SOH versus cycle number
- Analyze relationships between temperature, voltage, current and degradation

Phase 4 — Feature Engineering

- Identify which measurements make useful ML features
- Create statistical and derived features per cycle
- Actively avoid target leakage
- Define the final feature matrix X and target y

Phase 5 — Machine Learning

- Train a Linear Regression model as a baseline
- Train a Random Forest Regressor
- Train a Gradient Boosting Regressor

Phase 6 — Model Evaluation & Comparison

- Evaluate every model using MAE, RMSE and R^2
- Apply the same evaluation procedure to all three models

Phase 7 — Cross-Validation & Hyperparameter Tuning

- Use a validation strategy appropriate for sequential, cycle-ordered data
- Avoid data leakage between training and validation sets
- Perform cross-validation on the best-performing model
- Tune hyperparameters and compare performance before and after tuning

Phase 8 — Final SOH Prediction

- Train the final chosen model on the full training data
- Predict SOH on unseen (held-out) data
- Classify battery condition from the predicted SOH
- Analyze where and why prediction errors occur

Phase 9 — RUL Estimation

- Analyze the SOH degradation curve produced by the final model
- Define an End-of-Life (EOL) threshold
- Estimate the cycle at which EOL is reached
- Estimate remaining cycles from the current cycle to EOL
- Optionally convert remaining cycles into an approximate time estimate

Phase 10 — Final Analysis

- Summarize final metrics across all models
- Present final graphs and visual results
- Report feature importance for the final model
- Summarize results, limitations and future improvements

6. Overall System Architecture

The diagram below shows the complete end-to-end pipeline of the project, from raw dataset to final RUL results. Each stage feeds directly into the next.

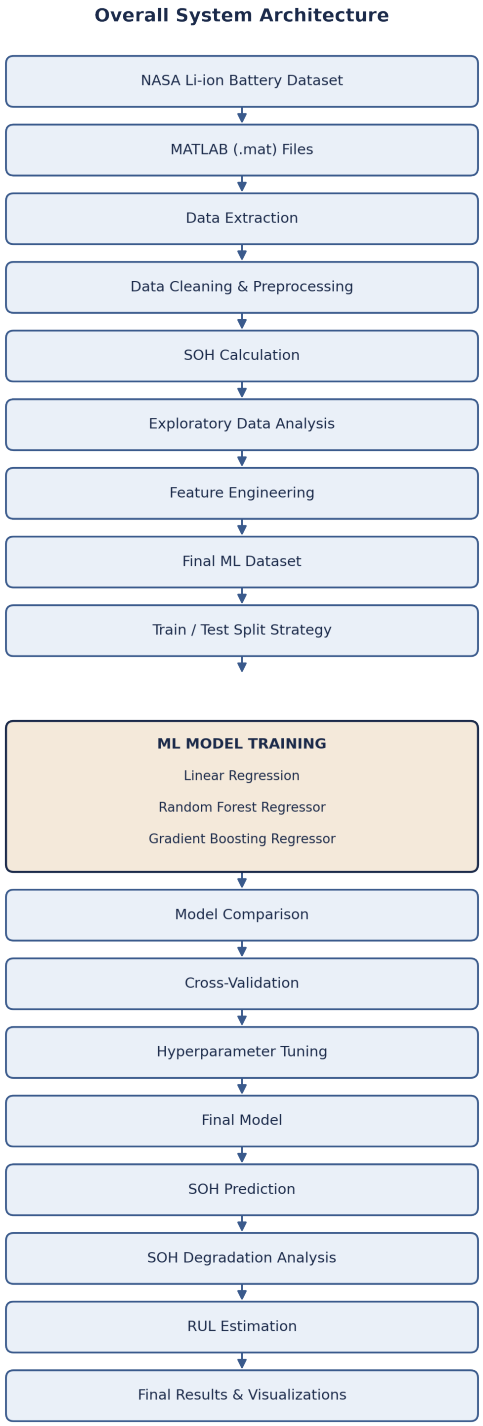


Figure 6.1 — Overall system architecture, from raw dataset to final RUL output.

7. Detailed ML Architecture

This diagram focuses specifically on the machine learning portion of the pipeline — how raw battery measurements become a trained, tuned model that outputs a predicted SOH value.

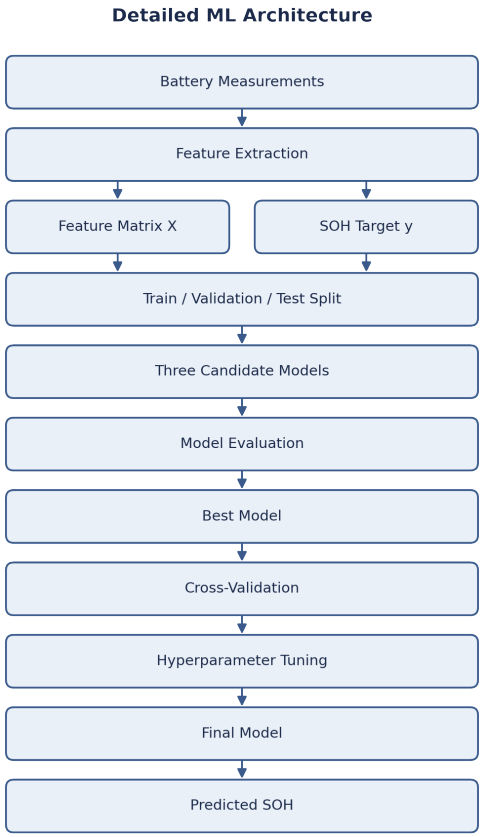


Figure 7.1 — Detailed ML architecture from feature extraction to predicted SOH.

What X and y Represent

X (Feature Matrix) — the set of independent, per-cycle input variables derived from battery measurements (e.g. cycle number, voltage statistics, current statistics, temperature statistics, and charge/discharge duration). Each row of X corresponds to one battery cycle.

y (Target Vector) — the SOH (%) value calculated for that same cycle, using the capacity-based formula defined in Section 9. The model is trained to learn the mapping from X to y.

8. Input and Output Architecture

Input Features

Only features that can actually be extracted or derived from the NASA dataset are used. Current capacity is deliberately excluded as a direct input because SOH is calculated from capacity — including it directly would cause target leakage (see Section 15).

Candidate Input Feature	Description
Cycle number	The index of the charge/discharge cycle
Voltage statistics	e.g. mean, min, max, or slope of voltage during charge/discharge
Current statistics	e.g. mean, min, max current applied during charge/discharge
Temperature statistics	e.g. mean, min, max cell temperature during a cycle
Charge/discharge duration	Time taken to complete the charge or discharge phase
Energy-related features	Derived quantities such as approximate energy transferred in a cycle

Target

SOH (%) — State of Health, calculated as described in Section 9.

Final Outputs

- Predicted SOH (%)
- Battery health category (derived from predicted SOH)
- SOH degradation trend (visualized over cycles)
- Estimated End-of-Life (EOL) cycle
- Estimated Remaining Useful Life (RUL), in cycles

9. SOH Calculation Architecture

SOH is calculated directly from measured battery capacity:

$$\text{SOH} = (\text{Current Capacity} / \text{Initial Capacity}) \times 100$$

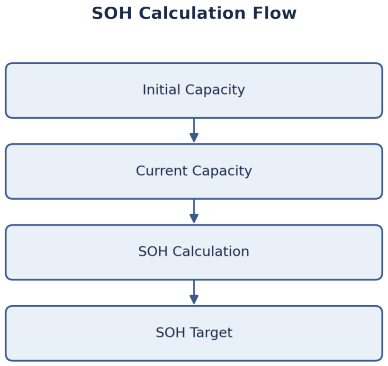


Figure 9.1 — SOH calculation flow, from capacity measurements to SOH target.

Numerical Example

If a cell's initial (rated) capacity is 2.0 Ah and its measured capacity at a later cycle is 1.6 Ah, then:

$$\text{SOH} = (1.6 / 2.0) \times 100 = 80\%$$

This means the battery retains 80% of its original capacity at that cycle. This example uses illustrative numbers only; actual capacity values will be taken from the dataset during Phase 3.

Because capacity is used to construct the SOH target itself, it must not also be used as an input feature for the ML models — doing so would let the model "see" the answer directly rather than learning to predict it from independent usage data.

10. RUL Architecture

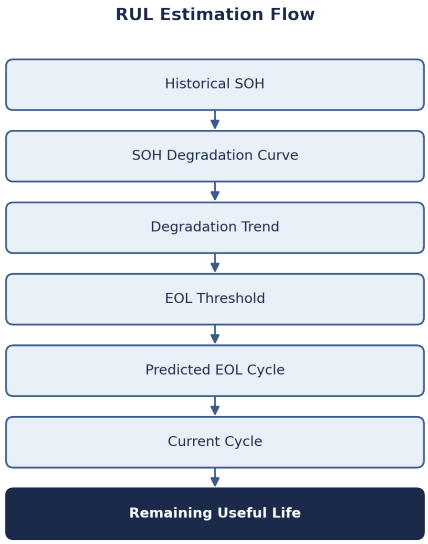


Figure 10.1 — RUL estimation flow, from historical SOH to remaining useful life.

Once the SOH degradation trend is established, RUL is estimated as:

RUL = Predicted EOL Cycle − Current Cycle

The End-of-Life (EOL) threshold is a chosen SOH percentage (commonly 70-80%) below which the battery is considered to have reached the end of its useful life. The predicted EOL cycle is the point at which the fitted degradation trend is expected to cross this threshold.

*RUL is an **estimate**, not a guarantee. It depends on the battery continuing to be used under conditions similar to those seen in the historical data — a genuine change in future operating conditions (temperature, charging behavior, load) can shift the actual end-of-life point earlier or later than predicted.*

11. Notebook Architecture

Implementation will later be organized into eight sequential Jupyter notebooks, matching the ten project phases from Section 5. Each notebook has a single, well-scoped responsibility.

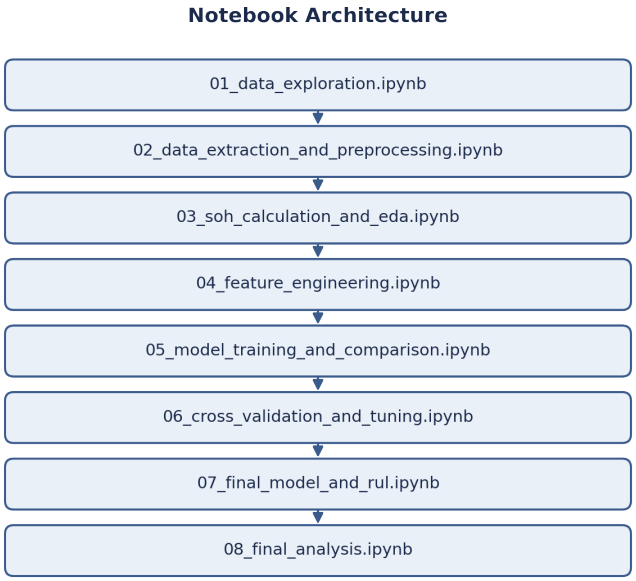


Figure 11.1 — Planned notebook sequence.

01_data_exploration.ipynb

Notebook	Detail
Purpose	Understand the raw dataset structure.
Inputs	Raw .mat files for B0005, B0006, B0007, B0018.
Main Operations	Load .mat files; inspect nested structure; identify cycle types and fields.
Outputs	A clear map of the data structure (no cleaned dataset yet).
Should NOT Include	Do not extract or transform data here — inspection only.

02_data_extraction_and_preprocessing.ipynb

Notebook	Detail
Purpose	Convert raw nested data into a usable tabular dataset.
Inputs	Raw .mat files.
Main Operations	Extract charge/discharge records; flatten nested structure; handle missing values; merge batteries.
Outputs	A combined, cleaned tabular dataset (all batteries, all cycles).
Should NOT Include	Do not calculate SOH or engineer ML features here.

03_soh_calculation_and_eda.ipynb

Notebook	Detail
Purpose	Calculate SOH and understand degradation patterns.
Inputs	Cleaned tabular dataset from Notebook 02.
Main Operations	Calculate SOH per cycle; plot SOH vs cycle; explore voltage/current/temperature relationships.
Outputs	Dataset with SOH column added; EDA plots.
Should NOT Include	Do not build ML features or train models here.

04_feature_engineering.ipynb

Notebook	Detail
Purpose	Build the ML-ready feature set.
Inputs	SOH dataset from Notebook 03.
Main Operations	Derive statistical/aggregated features per cycle; assemble X; assemble y; check for leakage.
Outputs	Final ML dataset (X, y) ready for modeling.
Should NOT Include	Do not use current capacity directly as a feature.

05_model_training_and_comparison.ipynb

Notebook	Detail
Purpose	Train and compare the three candidate models.
Inputs	Final ML dataset from Notebook 04.
Main Operations	Train Linear Regression, Random Forest and Gradient Boosting; evaluate with MAE/RMSE/R^2.
Outputs	A model comparison table.
Should NOT Include	Do not tune hyperparameters here — use default/basic settings for a fair baseline comparison.

06_cross_validation_and_tuning.ipynb

Notebook	Detail
Purpose	Validate and tune the best-performing model.
Inputs	Final ML dataset; best model from Notebook 05.
Main Operations	Apply a cycle-aware validation strategy; cross-validate; tune hyperparameters; compare before/after.
Outputs	A tuned final model and validated performance metrics.
Should NOT Include	Do not introduce random shuffling that ignores cycle order.

07_final_model_and_rul.ipynb

Notebook	Detail
Purpose	Produce final SOH predictions and estimate RUL.
Inputs	Tuned final model from Notebook 06.
Main Operations	Predict SOH on held-out data; classify battery condition; fit degradation trend; estimate EOL and RUL.
Outputs	Final SOH predictions, degradation plot, EOL/RUL estimates.
Should NOT Include	Do not retrain alternative models here.

08_final_analysis.ipynb

Notebook	Detail
Purpose	Summarize the complete project outcome.
Inputs	All outputs from Notebooks 05-07.
Main Operations	Consolidate metrics, plots and feature importance; discuss limitations and future work.
Outputs	Final summarized results for the project report.
Should NOT Include	Do not introduce new modeling steps here.

12. Technology Stack

Technology	Purpose
Python	Core programming language for the entire pipeline.

Technology	Purpose
NumPy	Numerical array operations and mathematical computations.
Pandas	Tabular data handling, cleaning and transformation.
SciPy	Scientific computing utilities, including signal/statistics support for degradation analysis.
Matplotlib	Base plotting library for graphs and visualizations.
Seaborn	Statistical visualization built on Matplotlib, used for EDA plots.
Scikit-learn	Machine learning models, evaluation metrics, cross-validation and tuning utilities.
Joblib	Saving and loading the final trained model (.pkl file).

Deep learning frameworks and web technologies are intentionally excluded from the current scope (see Section 17).

13. Model Comparison Strategy

Three regression models are used so that model choice is based on evidence rather than assumption. All three are trained on the same feature set and evaluated using the same metrics, under the same validation strategy.

Linear Regression

Used as a simple, interpretable baseline. It establishes the minimum performance bar that more complex models should improve upon.

Random Forest Regressor

An ensemble of decision trees used to capture nonlinear relationships between usage features and SOH that a linear model cannot represent.

Gradient Boosting Regressor

A second, boosting-based nonlinear approach, used as an alternative to Random Forest for capturing complex degradation patterns.

No model is assumed to be best in advance — the final choice is made strictly on the basis of the comparison performed in Phase 6.

14. Evaluation Metrics

MAE — Mean Absolute Error

The average absolute difference between predicted and actual SOH values. It is easy to interpret directly in SOH percentage units. Lower MAE = better.

RMSE — Root Mean Squared Error

Similar to MAE but squares errors before averaging, which penalizes larger errors more heavily. Useful for highlighting models that make occasional large mistakes. Lower RMSE = better.

R^2 — Coefficient of Determination

Describes how much of the variation in SOH is explained by the model, on a scale up to 1.0. Higher R^2 = better.

15. Data Leakage Prevention

This section is deliberately treated as a first-class design concern, because battery-cycle data has properties that make leakage easy to introduce accidentally.

Why Battery Data Is Sequential

Each cycle happens after the one before it, and a battery's condition at cycle N is physically dependent on everything that happened in cycles 1 through N-1. This ordering is meaningful information, not an arbitrary row index.

Why Random Splitting Can Cause Leakage

If cycles are split into train and test sets purely at random, the model can end up training on cycles that occur *after* a test cycle in real time. This lets the model implicitly learn from future degradation, producing evaluation scores that look better than the model would actually achieve in a real, forward-in-time deployment.

Why Future Cycles Should Not Influence Training

A model intended to predict SOH going forward must only ever be trained on information that would genuinely have been available at that point in time. Allowing later cycles to influence training defeats the purpose of the RUL estimation that follows.

Why Validation Must Respect Battery-Cycle Structure

The validation strategy chosen in Phase 7 must preserve cycle order (for example, training on earlier cycles and validating on later ones) rather than shuffling cycles randomly, so that reported performance reflects realistic forward-in-time prediction.

Why Current Capacity Should Not Be a Direct Input

Since SOH is calculated directly from capacity (Section 9), using current capacity as an input feature would let the model trivially reconstruct the target instead of learning the underlying usage-to-degradation relationship. This is a direct form of target leakage and is excluded by design in Section 8.

16. Expected Final Project Flow

A simplified, one-page view of the complete project from start to finish:

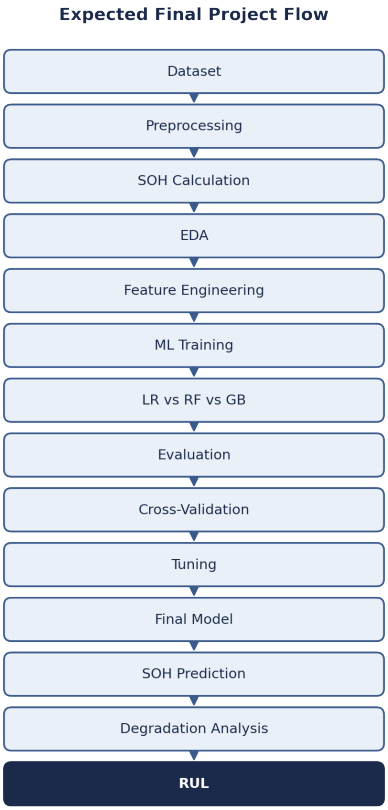


Figure 16.1 — Simplified end-to-end project flow.

17. Project Scope

Included

- NASA battery dataset
- Four battery cells (B0005, B0006, B0007, B0018)
- Data preprocessing
- SOH calculation
- Exploratory data analysis
- Feature engineering
- Scikit-learn regression models
- Model comparison
- Cross-validation
- Hyperparameter tuning
- SOH prediction
- RUL estimation
- Visualizations

Not Included Currently

- Physical EV hardware
- Real Battery Management System (BMS) integration
- CAN bus interfacing
- Real-time sensor acquisition
- React / FastAPI / Streamlit or any web development
- Cloud deployment
- Deep learning models

Items in the "Not Included" list are noted only as possible future extensions (Section 19) and are not part of the current implementation architecture.

18. Final Deliverables

- Clean, combined battery dataset
- SOH-labeled dataset
- EDA graphs
- Feature-engineered ML dataset (X, y)
- Three trained candidate models (Linear Regression, Random Forest, Gradient Boosting)
- Model comparison table (MAE, RMSE, R^2)
- Cross-validation results
- Tuned final model
- Final SOH predictions on unseen data
- SOH degradation plots
- RUL estimation results
- Saved final model file (.pkl)
- Complete set of Jupyter notebooks (01 through 08)
- Final project report

19. Future Enhancements

The following are possible directions for future work beyond the current scope. None of these are implemented as part of this project.

- Real-time BMS integration
- Real EV sensor data (beyond the NASA laboratory dataset)
- CAN bus data ingestion
- Real-time battery monitoring
- Deep learning models (e.g. LSTM/temporal architectures)
- Cloud deployment
- Web dashboard for visualization
- Cell-level SOH prediction at pack scale